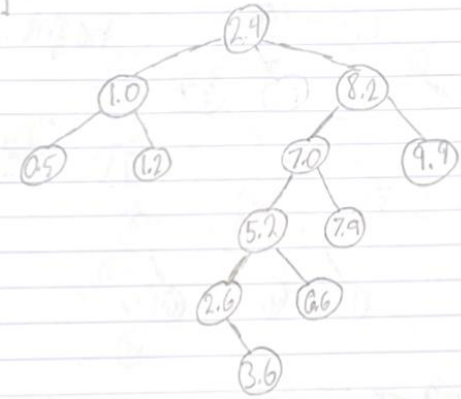
1)

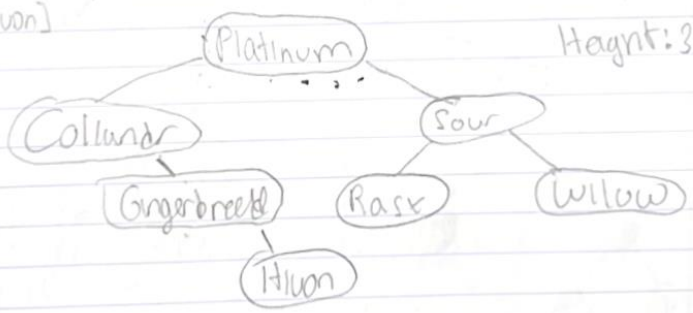
1)

a) [2.4, 8.2, 7.0, 5.2, 1.0, 6.5, 2.6, 3.6, 7.9, 0.5, 6.6, 9.9, 1.2]



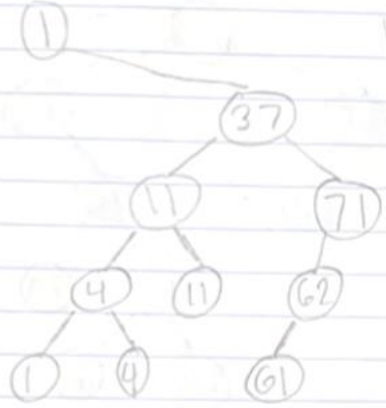
Height: 5

b) [Platinum, Coliander, Sour, Willow, Rasx, Gingerbread, Hiwon]



Height: 3

c) [1, 37, 11, 71, 62, 4, 61, 1, 11, 4]



Height: 4

d) [78, 76, 56, 43, 55, 77, 10, 93, 46, 79, 33, 8, 89, 100, 98, 19]

Height: 6

2)

2

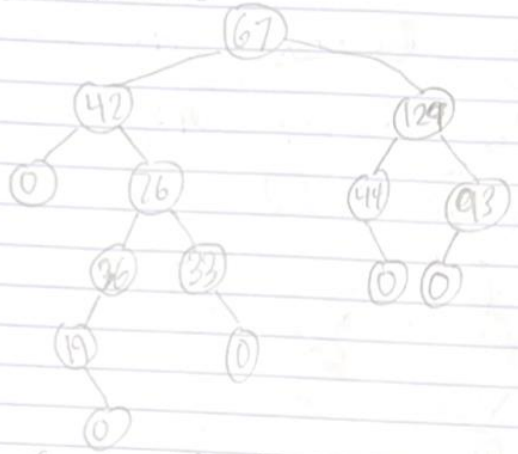
Original

Operations: node.data = left + (right/2)

Pre order: visit → left → right
(V) (L) (R)

V:68	L:21	R:92	21 + (92/2) = 67
V:21	L:15	R:54	15 + (54/2) = 42
V:15	L:0	R:0	= 0
V:54	L:49	R:59	49 + (59/2) = 76
V:49	L:36	R:0	36 + (0/2) = 36
V:36	L:0	R:37	0 + (37/2) = 19
V:37	L:0	R:0	= 0
V:59	L:0	R:65	0 + (65/2) = 33
V:65	L:0	R:0	= 0
V:92	L:80	R:97	80 + (97/2) = 129
V:80	L:0	R:87	0 + (87/2) = 44
V:87	L:0	R:0	= 0
V:97	L:93	R:0	93 + (0/2) = 93
V:93	L:0	L:0	0 + (0/2) = 0

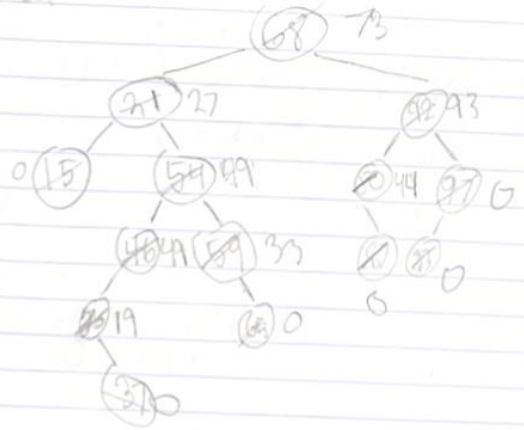
Tree after Pre Order



Preorder Question b) No its not a BST tree because there nodes where the right child is not greater than the node.

c) AVL has to be a BST and balanced height which this tree is not

Orginal

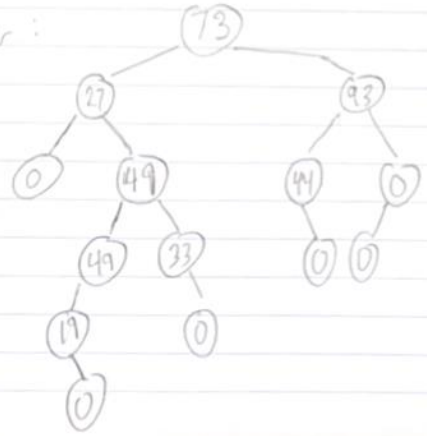


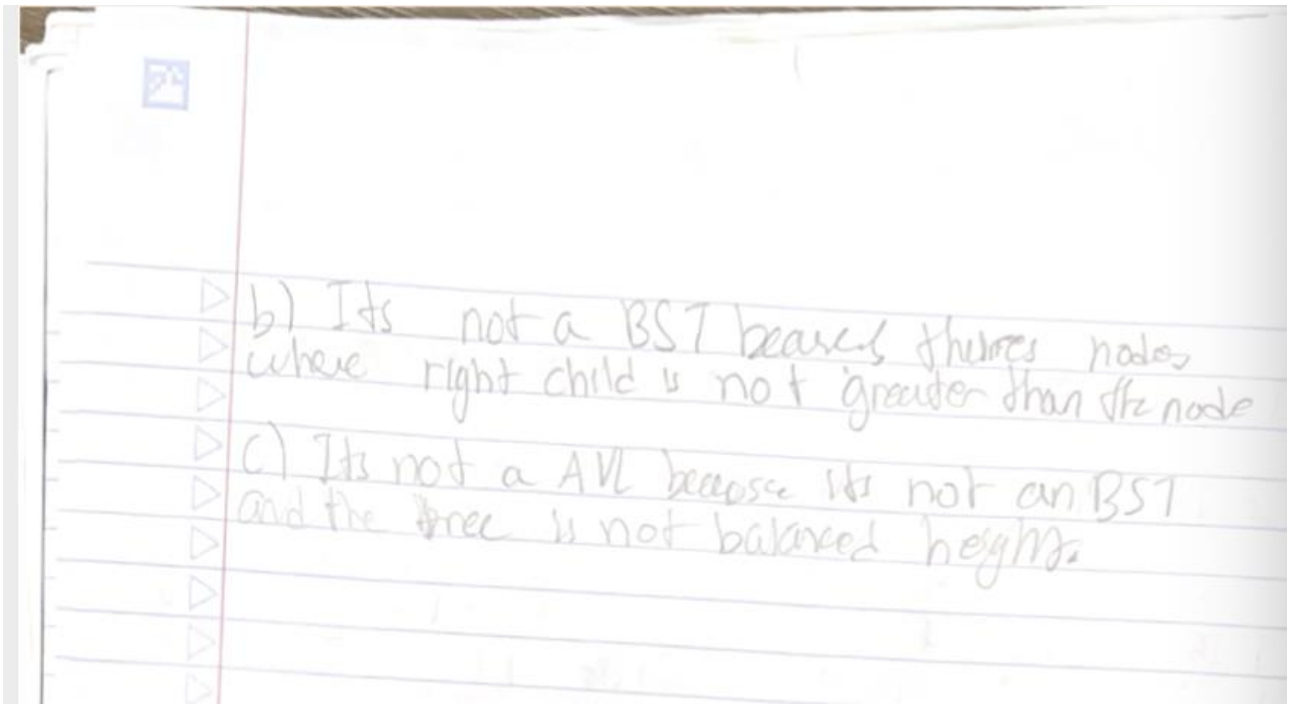
In order Tree Operation: node = left + (right /2)

In Order: Left → Visit → Right (Visit Least to greatest)
(L) (V) (R)

V:15	L:0	R:0	= 0
V:21	L:0	R:54	0+(54/2) = 27
V:36	L:0	R:37	0+(37/2) = 19
V:37	L:0	R:0	= 0
V:46	L:19	R:0	19+(0/2) = 19
V:54	L:19	R:59	19+(59/2) = 49
V:59	L:0	R:65	0+(65/2) = 33
V:65	L:0	R:0	= 0
V:68	L:27	R:92	27+(92/2) = 73
V:80	L:0	R:87	0+(87/2) = 44
V:87	L:0	R:0	0+(0/2) = 0
V:92	L:44	R:97	44+(97/2) = 93
V:93	L:0	R:0	0+(0/2) = 0
V:97	L:0	R:0	= 0

Tree after in Order:




b) Its not a BST because there nodes where right child is not greater than the node

c) Its not a AVL because its not an BST and the tree is not balanced height.

7)

Question 3:

I did not get a chance to fully code the Huffman coding problem, but I think can predict the time and space complexity.

The algorithm time complexity is O(n) where n is the length of the input string, because the number of distinct characters is always at most 27 (a through z and space) which is a constant, so the only thing that grows is the length of the input string.

The algorithm time complexity in the best case is Ω(n) because we always have to read every character in the string at least once to count the frequencies.

The algorithm space complexity is O(n) where n is the length of the input string, because the encoded output string grows with the size of the input and everything else like the frequency map and the heap are at most 27 entries which is constant.

Question 4:

The algorithm time complexity is O(n) where n is the number of nodes in the encrypted binary tree, because in the worst case every node of the tree must be visited to confirm the element does not exist.

The algorithm time complexity in the best case is Ω(1) because the element you're looking for is the root.

The algorithm space complexity is O(log n) where n is the number of nodes in the encrypted binary tree because no new data structures are being created so the only use of memory is used from the call stack from recursion. The stack grows as deep as the height of the tree and for binary trees the height is log n.

Question 5:

The algorithm time complexity for the best and worst case scenario is O(n) where N is the number of strings in the heap because you need to check all strings to see whose length is less than the input.

The algorithm space complexity is O(n) where n is the number of strings in the heap because the that's created array list can store up to n number of strings.

Question 6:
The algorithm time complexity is O(n + m log n) where n is the number of nodes in the red black tree and m is the number of nodes being deleted, because in the worst case we visit every node once during the collection phase which is O(n), and then for each of the m nodes we delete, each deletion takes O(log n) time since the tree rebalances after every delete.

The algorithm time complexity in the best case is Ω(n) because even if no nodes are in range, we still have to walk the entire tree during the collection phase to confirm nothing qualifies.

The algorithm space complexity is O(m + log n) where m is the number of nodes being deleted and n is the number of nodes in the tree, because the arrayist we create stores up to m keys, and the recursion stack during collection grows as deep as the height of the tree which is log n for a red black tree.